Enterprise DevSecOps Security Lab

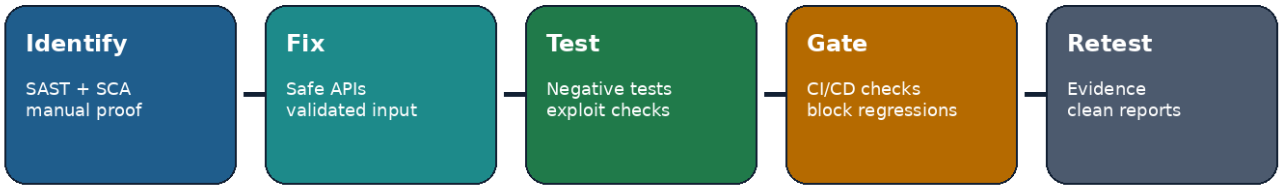
Secure-code Remediation Examples

Professional developer-focused remediation guide for the vulnerable Flask application findings, including unsafe code patterns, corrected implementations, tests, CI/CD gates, and retest criteria.

Author	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Document Type	Secure-code Remediation Examples
Primary App	Python Flask vulnerable application

Findings Covered	Secure Practices Demonstrated
SQL Injection	Parameterized queries
Command Injection	Input validation and allowlists
Template Injection / Unsafe Rendering	Safe template rendering
Dependency Vulnerabilities	Dependency upgrade workflow
	Regression tests and CI/CD gates

Secure Code Remediation Workflow



This document uses safe, educational examples based on the private lab only. The vulnerable snippets are included to explain remediation and should not be deployed to public environments.

1. Executive Overview

The vulnerable Flask application intentionally contains insecure code patterns that are useful for AppSec learning, CI/CD scanning, and remediation practice. This document translates those findings into secure development examples that can be used in a professional remediation package, pull request, or developer handoff.

Remediation Objective	Replace vulnerable patterns with safe framework-supported controls, add negative regression tests, and enforce the controls through CI/CD so the same issues cannot silently re-enter the codebase.

1.1 Findings-to-Remediation Summary

Finding	Unsafe Pattern	Secure Remediation	Validation Method
SQL Injection	String-formatted SQL query using request input	Parameterized query, integer validation, query helper function	SQLi payload returns 400 or one valid record only
Command Injection	shell=True with untrusted host parameter	Allowlist/strict validation, subprocess list args, shell=False, timeout	Injected command is rejected and never executed
Template Injection / Unsafe Rendering	User input becomes template source string	Render fixed templates and pass user input as context variable	Payload renders as text, not executable template logic
Dependency Vulnerability	Known vulnerable dependency pin	Upgrade, lock, audit, and fail CI on high-risk issues	pip-audit produces clean or accepted-risk output

1.2 Secure-code Principles Used

- Prefer safe framework primitives over custom string construction.
- Treat all request parameters as untrusted until validated and typed.
- Use allowlists when the expected input space is small and predictable.
- Fail safely with generic error messages and appropriate HTTP status codes.
- Add exploit regression tests before closing the finding.
- Make CI/CD enforce the fixed standard using SAST, SCA, and unit tests.

2. Current Vulnerable Code Surface

The reviewed application file is src/app/app.py. The three security-sensitive routes are shown below. Each route accepts untrusted user input and uses it in a risky sink.

Route	Input Parameter	Risky Sink	Primary Risk
/user	id	sqlite cursor.execute(query)	SQL injection and unauthorized data retrieval
/ping	host	subprocess.run(..., shell=True)	Operating system command injection
/hello	name	render_template_string(template)	Unsafe server-side template rendering pattern

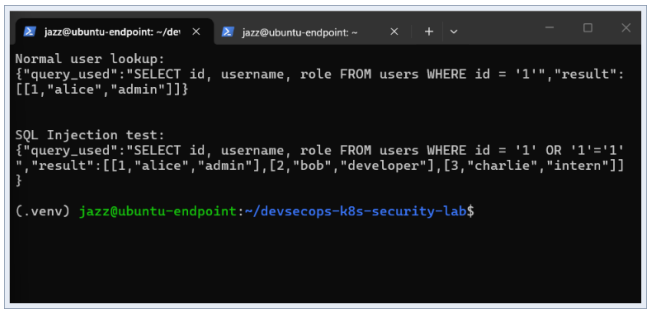
Vulnerable SQL route

```
@app.route("/user")
def get_user():
    user_id = request.args.get("id", "1")
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    query = f"SELECT id, username, role
        FROM users WHERE id = '{user_id}'"
    result =
        cursor.execute(query).fetchall()
    return {"query_used": query, "result":
         result}
```

Vulnerable command route

```
@app.route("/ping")
def ping():
    host = request.args.get("host",
        "127.0.0.1")
    result = subprocess.run(
        f"ping -c 1 {host}",
        shell=True,
        capture_output=True,
        text=True
    )
    return f"<pre>{result.stdout}</pre>"
```

2.1 Evidence References



SQL injection proof from private lab evidence.

Command injection proof from private lab evidence.

3. SQL Injection Remediation Example

The SQL injection issue occurs because user-controlled input is concatenated into a SQL statement. The secure solution is not to blacklist characters; the secure solution is to separate query structure from query data using parameterized queries.

Secure Coding Rule	Never build SQL statements by inserting request parameters into a query string. Validate and type the input, then pass it to the database driver as a parameter.

Before: unsafe string interpolation

```
user_id = request.args.get("id", "1")
query = f"SELECT id, username, role FROM
    users WHERE id = '{user_id}'"
result = cursor.execute(query).fetchall()
```

After: typed input + parameterized query

```
from flask import abort

def parse_user_id(value: str) -> int:
    if value is None or not
        value.isdigit():
        abort(400, description="Invalid
            user id")
    return int(value)

user_id =
    parse_user_id(request.args.get("id"))
cursor.execute(
    "SELECT id, username, role FROM users
        WHERE id = ?",
    (user_id,)
)
result = cursor.fetchall()
```

3.1 Secure Route Implementation

Recommended fixed implementation

```
@app.route("/user")
def get_user():
    user_id = parse_user_id(request.args.get("id"))

    with sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True) as conn:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT id, username, role FROM users WHERE id = ?",
            (user_id,)
        )
        rows = cursor.fetchall()

    return {"result": rows}
```

3.2 Why This Fix Works

- The database driver treats user_id as data, not executable SQL syntax.
- The route rejects non-numeric identifiers before the query is executed.
- The response no longer exposes the raw query, reducing information disclosure.
- The read-only SQLite URI remains compatible with the hardened container filesystem design.

4. SQL Injection Regression Tests

A remediation should not be accepted only because the code looks safer. The exploit must be retested and converted into a regression test so future changes cannot reintroduce the issue silently.

pytest examples for SQL injection remediation

```
def test_user_lookup_accepts_valid_numeric_id(client):
    response = client.get("/user?id=1")
    assert response.status_code == 200
    assert b"alice" in response.data


def test_user_lookup_rejects_sql_injection_payload(client):
    payload = "1' OR '1'='1"
    response = client.get(f"/user?id={payload}")
    assert response.status_code == 400


def test_user_lookup_rejects_non_numeric_id(client):
    response = client.get("/user?id=abc")
    assert response.status_code == 400
```

4.1 Manual Retest Commands

Manual validation commands

```
# Expected: 200 OK and a single authorized record
curl "http://localhost:5000/user?id=1"

# Expected after fix: 400 Bad Request, no database enumeration
curl "http://localhost:5000/user?id=1'%20OR%20'1'%3D'1"

# Expected after fix: 400 Bad Request
curl "http://localhost:5000/user?id=abc"
```

4.2 Acceptance Criteria

- No SQL payload returns multiple users or changes query logic.
- Invalid IDs return 400 Bad Request with a generic message.
- Valid IDs continue to return expected application data.
- Bandit/manual review no longer identifies string-built SQL execution in the route.

5. Command Injection Remediation Example

The command injection issue occurs because untrusted input is executed through the operating system shell. The fixed implementation must avoid shell interpretation entirely and should validate the target before invoking a system utility.

Secure Coding Rule	Do not pass request input to a shell command. Use argument arrays, shell=False, strict validation, timeout controls, and generic error handling.

Before: shell command constructed from user input

```
host = request.args.get("host",
    "127.0.0.1")
result = subprocess.run(
    f"ping -c 1 {host}",
    shell=True,
    capture_output=True,
    text=True
)
```

After: validated input + shell disabled

```
host =
    validate_host(request.args.get("host")
    )
result = subprocess.run(
    ["ping", "-c", "1", host],
    shell=False,
    capture_output=True,
    text=True,
    timeout=3,
    check=False
)
```

5.1 Safer Host Validation Helper

Host validation helper

```
import ipaddress
import re
from flask import abort

DOMAIN_RE = re.compile(r"^(?=.{1,253}$)([a-zA-Z0-9-]{1,63}\.)*[a-zA-Z0-9-]{1,63}$")
ALLOWED_LOCAL_TARGETS = {"127.0.0.1", "localhost"}


def validate_host(value: str) -> str:
    if not value:
        abort(400, description="Missing host")

    value = value.strip()

    if value in ALLOWED_LOCAL_TARGETS:
        return value

    try:
        return str(ipaddress.ip_address(value))
    except ValueError:
        pass

    if DOMAIN_RE.fullmatch(value):
        return value

    abort(400, description="Invalid host")
```

6. Command Injection Secure Route and Tests

Recommended fixed /ping route

```
@app.route("/ping")
def ping():
    host = validate_host(request.args.get("host", "127.0.0.1"))

    try:
        result = subprocess.run(
            ["ping", "-c", "1", host],
            shell=False,
            capture_output=True,
            text=True,
            timeout=3,
            check=False
        )
    except subprocess.TimeoutExpired:
        abort(504, description="Ping timed out")

    return {"host": host, "returncode": result.returncode, "output":
        result.stdout[:2000]}
```

6.1 Regression Tests

pytest examples for command injection remediation

```
def test_ping_allows_localhost(client):
    response = client.get("/ping?host=127.0.0.1")
    assert response.status_code == 200


def test_ping_rejects_command_separator(client):
    response = client.get("/ping?host=127.0.0.1;whoami")
    assert response.status_code == 400


def test_ping_rejects_shell_substitution(client):
    response = client.get("/ping?host=$(whoami)")
    assert response.status_code == 400


def test_ping_rejects_pipe_operator(client):
    response = client.get("/ping?host=127.0.0.1|id")
    assert response.status_code == 400
```

6.2 Manual Retest Commands

Manual validation commands

```
# Expected: 200 OK
curl "http://localhost:5000/ping?host=127.0.0.1"

# Expected after fix: 400 Bad Request, whoami is never executed
curl "http://localhost:5000/ping?host=127.0.0.1%3Bwhoami"

# Expected after fix: 400 Bad Request
curl "http://localhost:5000/ping?host=%24(whoami)"
```

7. Template Injection / Unsafe Rendering Remediation

The unsafe rendering pattern occurs when user input is inserted into a template string and that string is then interpreted by the template engine. In Flask/Jinja, the safer pattern is to render a fixed template and pass user input as a variable.

Secure Coding Rule	User input should be data inside a template context, not part of the template source code.

Before: user input becomes template source

```
@app.route("/hello")
def hello():
    name = request.args.get("name",
        "guest")
    template = f"<h1>Hello {name}</h1>"
    return
        render_template_string(template)
```

After: fixed template + context variable

```
from flask import render_template

@app.route("/hello")
def hello():
    name = sanitize_display_name(request.a
        rgs.get("name", "guest"))
    return render_template("hello.html",
        name=name)
```

7.1 Secure Display-name Helper

Display input validation helper

```
import re
from flask import abort

NAME_RE = re.compile(r"^[A-Za-z0-9 ._-]{1,40}$")


def sanitize_display_name(value: str) -> str:
    value = (value or "guest").strip()
    if not NAME_RE.fullmatch(value):
        abort(400, description="Invalid display name")
    return value
```

7.2 Template File Example

Safe template file

```
<!-- templates/hello.html -->
<!doctype html>
<html lang="en">
  <head><title>Hello</title></head>
  <body>
    <h1>Hello {{ name }}</h1>
  </body>
</html>
```

8. Template Rendering Tests and Dependency Fixes

Unsafe rendering fixes should prove that template expressions are not evaluated when submitted through request parameters. Dependency remediation should prove that vulnerable packages have been upgraded and remain audited in CI/CD.

8.1 Template Regression Tests

pytest examples for unsafe rendering remediation

```
def test_hello_allows_safe_name(client):
    response = client.get("/hello?name=Jagriti")
    assert response.status_code == 200
    assert b"Hello Jagriti" in response.data


def test_hello_rejects_template_expression(client):
    response = client.get("/hello?name={{7*7}}")
    assert response.status_code == 400


def test_hello_rejects_html_script_payload(client):
    response = client.get("/hello?name=<script>alert(1)</script>")
    assert response.status_code == 400
```

8.2 Dependency Remediation Example

Before: vulnerable dependency pin

```
# src/app/requirements.txt
Flask==2.3.3
requests==2.19.1
gunicorn==21.2.0
```

After: audited and maintained pins

```
# src/app/requirements.txt
Flask==3.1.1
requests==2.32.4
gunicorn==23.0.0

# Confirm exact versions with pip-audit in
     CI before release.
```

8.3 Dependency Retest Commands

SCA retest and lock workflow

```
python -m pip install --upgrade pip
pip install -r src/app/requirements.txt
pip-audit -r src/app/requirements.txt

# Optional reproducibility step:
pip-compile --generate-hashes requirements.in
```

9. Consolidated Secure Flask Example

The following condensed implementation demonstrates how the remediated route patterns can coexist in one secure Flask application. This sample is intentionally concise for a developer handoff; production code should further separate routes, services, validators, and database helpers.

Secure app.py reference pattern

```
from flask import Flask, request, abort, render_template
import ipaddress, re, sqlite3, subprocess, os

app = Flask(__name__)
DB_PATH = os.path.join(os.path.dirname(__file__), "users.db")
NAME_RE = re.compile(r"^[A-Za-z0-9 ._-]{1,40}$")
DOMAIN_RE = re.compile(r"^(?=.{1,253}$)([a-zA-Z0-9-]{1,63}\.)*[a-zA-Z0-9-]{1,63}$")


def parse_user_id(value):
    if value is None or not value.isdigit():
        abort(400, description="Invalid user id")
    return int(value)


def sanitize_display_name(value):
    value = (value or "guest").strip()
    if not NAME_RE.fullmatch(value):
        abort(400, description="Invalid display name")
    return value


def validate_host(value):
    value = (value or "127.0.0.1").strip()
    if value in {"127.0.0.1", "localhost"}:
        return value
    try:
        return str(ipaddress.ip_address(value))
    except ValueError:
        if DOMAIN_RE.fullmatch(value):
            return value
    abort(400, description="Invalid host")


@app.route("/user")
def get_user():
    user_id = parse_user_id(request.args.get("id"))
    with sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT id, username, role FROM users WHERE id = ?", (user_id,))
        return {"result": cursor.fetchall()}


@app.route("/ping")
def ping():
    host = validate_host(request.args.get("host"))
    result = subprocess.run(["ping", "-c", "1", host], shell=False,
                            capture_output=True, text=True, timeout=3, check=False)
    return {"host": host, "returncode": result.returncode, "output":
        result.stdout[:2000]}


@app.route("/hello")
def hello():
    name = sanitize_display_name(request.args.get("name", "guest"))
    return render_template("hello.html", name=name)
```

10. Security Test Plan for Remediated Code

A professional remediation package should include tests that demonstrate both normal functionality and malicious payload rejection. The test suite should be part of the pipeline, not a one-time manual activity.

Test ID	Scenario	Payload	Expected Result
SCT-001	Valid user lookup	/user?id=1	200 OK; one valid record
SCT-002	SQL injection attempt	/user?id=1' OR '1'='1	400 Bad Request
SCT-003	Command separator attempt	/ping?host=127.0.0.1;whoami	400 Bad Request
SCT-004	Command substitution attempt	/ping?host=$(id)	400 Bad Request
SCT-005	Template expression attempt	/hello?name={{7*7}}	400 Bad Request or literal text only
SCT-006	Dependency audit	pip-audit -r requirements.txt	No unresolved high-risk findings

10.1 Recommended pytest Fixture

Flask test client fixture

```
import pytest
from src.app.app import app

@pytest.fixture
def client():
    app.config.update(TESTING=True)
    with app.test_client() as client:
        yield client
```

10.2 Developer Definition of Done

- All exploit regression tests pass locally and in CI/CD.
- Bandit findings are resolved or explicitly accepted with documented rationale.
- pip-audit reports no unresolved vulnerable package pins above accepted risk threshold.
- Manual curl retests match expected safe behavior.
- The remediation branch includes code changes, tests, and updated evidence screenshots.

11. CI/CD Security Gates for Secure-code Enforcement

Secure code examples become more valuable when the pipeline enforces them. The following workflow snippets show how to make insecure code changes visible before merge.

11.1 Bandit Gate

GitHub Actions Bandit gate

```
- name: Run Bandit SAST
  run: |
    mkdir -p security/reports
    bandit -r src/app -f json -o security/reports/bandit-report.json
    bandit -r src/app
```

11.2 pip-audit Gate

GitHub Actions dependency gate

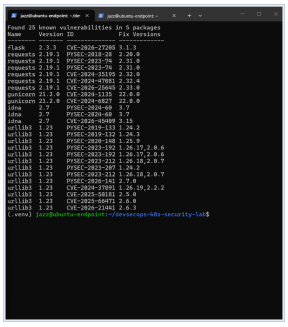
```
- name: Run pip-audit dependency scan
  run: |
    mkdir -p security/reports
    pip-audit -r src/app/requirements.txt       -f json -o security/reports/pip-audit-
        report.json
    pip-audit -r src/app/requirements.txt
```

11.3 Unit and Security Regression Tests

GitHub Actions exploit regression test gate

```
- name: Run application security regression tests
  run: |
    python -m pip install -r src/app/requirements.txt
    python -m pip install pytest
    pytest tests/security -v
```

Bandit SAST evidence from the lab.



pip-audit dependency evidence from the lab.

12. Secure Remediation Backlog and Mapping

The table below can be used in Jira, GitHub Issues, or a remediation tracker. It converts findings into developer tasks with security acceptance criteria.

Task ID	Developer Task	Security Acceptance Criteria	Priority
REM-001	Replace SQL f-string query with parameterized query	SQLi payload rejected; valid IDs still work	Critical
REM-002	Add user ID parser and 400 error handling	Non-numeric IDs rejected before DB access	High
REM-003	Replace shell command string with subprocess args list	shell=False; command separator payload rejected	Critical
REM-004	Add host allowlist / strict validator	Only valid IP/domain/local targets accepted	High
REM-005	Replace render_template_string with render_template	Template payload is not evaluated	High
REM-006	Upgrade vulnerable dependencies and audit in CI	pip-audit returns clean or documented accepted risk	High
REM-007	Add pytest exploit regression tests	All known exploit payloads are permanently covered	High

12.1 OWASP and CWE Reference Mapping

Finding	CWE Mapping	OWASP Top 10 Area	ASVS Theme
SQL Injection	CWE-89	Injection	Input validation, output encoding, data protection
Command Injection	CWE-78	Injection	Input validation and secure execution
Template Injection / Unsafe Rendering	CWE-94 / CWE-1336	Injection	Server-side input handling and safe rendering
Dependency Vulnerability	CWE-1104	Vulnerable and Outdated Components	Dependency and build integrity controls

13. Secure Pull Request Examples

The following examples can be pasted into a remediation pull request description to clearly show what changed and why the change reduces risk.

13.1 Pull Request Summary

Developer-friendly PR description

```
Title: Remediate Flask input handling and dependency vulnerabilities

Summary:
- Replaced SQL string interpolation with parameterized SQLite queries.
- Replaced shell command execution with validated input and shell=False.
- Replaced dynamic template rendering with fixed template files.
- Added exploit regression tests for SQLi, command injection, and unsafe rendering.
- Upgraded vulnerable Python dependencies and added pip-audit validation.

Security impact:
- Prevents database query manipulation through /user.
- Prevents OS command chaining through /ping.
- Prevents user input from being interpreted as template source.
- Reduces dependency exposure through audited package versions.
```

13.2 Example Commit Messages

Suggested commit message set

```
fix(appsec): parameterize user lookup query and validate user id
fix(appsec): disable shell execution in ping route
fix(appsec): replace unsafe template string rendering
fix(deps): upgrade vulnerable Python packages and audit requirements
test(appsec): add exploit regression tests for injection findings
ci(security): enforce bandit pip-audit and pytest gates
```

13.3 Reviewer Checklist

- Does the diff remove the vulnerable sink rather than only filtering a few characters?
- Are malicious payloads included as tests?
- Are errors safe and generic?
- Are dependencies pinned to audited versions?
- Do CI artifacts include SAST/SCA and regression test evidence?

14. Final Retest Checklist

The remediation is considered complete only after code review, automated security tests, manual exploit retesting, and report evidence updates are complete.

Control Area	Retest Activity	Pass Condition
SQL Injection	Run SQLi payloads against /user	No payload changes query logic; invalid input rejected
Command Injection	Run command separator, pipe, and substitution payloads against /ping	Payloads rejected; no OS command output appears
Template Rendering	Run Jinja expression and HTML/script payloads against /hello	Payloads rejected or rendered safely as data
Dependencies	Run pip-audit against requirements	No unresolved vulnerable package findings above accepted risk
SAST	Run Bandit against src/app	No high-confidence exploitable code findings remain
CI/CD	Push branch and inspect pipeline	Security gates pass and artifacts are stored
Documentation	Update finding status and evidence index	Report shows fixed status and retest evidence

14.1 Closing Statement

The secure-code examples in this document provide a clear remediation path for the vulnerable Flask application findings. They show how insecure patterns can be replaced with safer framework-supported controls, validated through tests, and enforced through the CI/CD pipeline. These examples are suitable for inclusion in a professional DevSecOps portfolio, remediation report, or developer handoff package.

Final Status Target	All remediation examples should be implemented in code, verified with exploit regression tests, scanned with Bandit and pip-audit, and documented with updated screenshots before the findings are marked as remediated.